Draft Outcome Prediction in League of Legends: A Hybrid Pipeline Combining Style Vectors, Collaborative Filtering, and Graph Neural Networks

Ayman Boussihmed Younès Mokhtari

Abstract

The draft phase in League of Legends is a critical strategic moment in which two teams alternately select champions from a pool of over 165 characters before each match, giving rise to a vast space of possible team compositions. Identifying effective synergies and counter-strategies within this space is a major challenge, further complicated by a game environment that is updated every two weeks. This study develops a hybrid pipeline that combines champion representation through style vectors, synergy modelling via popularity-corrected collaborative filtering embeddings, and team-level functional coverage analysis, evaluated against nine classifiers and a Graph Attention Network baseline. Models are trained on ranked match data and evaluated without retraining on professional match data from the 2026 LCK, LEC, and LCS seasons. Linear models systematically outperform non-linear alternatives, suggesting that the draft signal is real but predominantly additive in this context. The best models generalise to professional match data without retraining, demonstrating partial transferability of compositional patterns learned from ranked play.

Terminology:

Champion	A playable character, each with unique abilities and a distinct strategic role.
Draft phase	The pre-game phase in which two teams alternately pick and ban champions before a match.
Synergy	A performance advantage emerging when champions are played together, beyond their individual contributions.
Counter-pick	A champion selected to exploit a known weakness of an opponent's champion.
Meta	The set of champions and strategies considered most effective at a given time, shaped by periodic balance updates (patches) released every two weeks.
SoloQ	The solo ranked ladder, where players are matched individually rather than as a pre-formed team.
ADC	Attack Damage Carry — a marksman champion specialising in sustained ranged damage.
LCK / LEC / LCS	The premier professional leagues in Korea, Europe, and North America respectively.
Identity pick	A champion strongly associated with a specific team or player, used as part of a pre-planned strategy.

1 Introduction

Competitive League of Legends is structured around a draft phase in which two teams alternately select and ban champions before each match. This stage precedes the game itself and is widely recognised as a decisive factor in the match outcome. With a champion pool of over 165 champions, each with unique abilities, roles, and interaction patterns, the number of possible team compositions is vast. In the professional context, where individual matches can determine major qualifications, optimising the draft phase represents a key strategic and analytical challenge.

Understanding champion relationships requires tools capable of capturing the fundamentally relational nature of the data. Champions do not exist in isolation: the effectiveness of a composition emerges from subtle synergies between abilities, temporal power dynamics, and tactical roles. This relational complexity makes the draft well suited to graph-based modelling, where champions are represented as nodes and their interactions as edges.

Several approaches have been proposed to address the draft optimisation problem. Classical optimisation methods, such as MinMax algorithms combined with linear regression [1] or genetic algorithms [2], generate viable compositions but treat champions as independent entities, failing to capture interaction effects. Statistical approaches using association rule mining and k-means clustering [7] have identified recurring champion combinations, but require fixing the number of clusters arbitrarily and are limited to binary relationships. More targeted analyses, such as the identification of symbiotic relationships between marksman-support pairs in the bot lane [4], provide valuable insights into lane mechanics but do not scale to full five-champion team compositions. More recently, graph convolutional networks have been applied to champion recommendation [5], and random forest models have achieved high predictive accuracy on post-draft features [8], yet these models remain difficult to interpret: knowing that a composition has a 70% win probability does not indicate which synergies are activated or which champion should be added to improve these odds.

None of these approaches, however, integrate clustering, graph modelling, and neural learning into a unified framework. Parameters are estimated independently for each component, ignoring the structural coherence that links champion archetypes, relational patterns, and team-scale dynamics. Explainability, in particular, is the greatest challenge: for coaches and analysts to adopt automated systems, they must be able to understand and validate the underlying reasoning.

The aim of this study is therefore to develop a hybrid pipeline combining champion style vectors, popularity-corrected collaborative filtering embeddings, and team-level functional coverage analysis to model champion relationships and predict match outcomes from the draft phase, with a Graph Attention Network as a structural baseline and an evaluation of generalisation to professional play.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 introduces the theoretical background in graph theory, spectral analysis, and clustering. Section 4 details our methodology. Section 5 presents experimental results, and Section 7 concludes with directions for future work.

2 Related Work

2.1 Artificial Intelligence in Esports

2.1.1 Overview of Recent Work (2018–2025)

Artificial intelligence has made remarkable strides in esports over the past seven years, reshaping how athletes are evaluated and trained. This shift is directly linked to the fact that every esports match

generates massive quantities of exploitable data through game programming interfaces. Esports is characterised by its tactical depth, requiring rapid decisions under incomplete information and involving multiple, complex interactions.

The integration of AI in esports rests on three essential pillars. Predictive analytics seeks to forecast match outcomes from historical performance data, using approaches such as logistic regression or recurrent neural networks. Pre-match predictions achieve accuracies of 60% to 75%, while in-game predictions based on LSTM architectures can exceed 85% during the opening minutes of play. Performance analysis helps professional teams clearly identify their players' strengths and weaknesses, while shedding light on effective champion combinations and winning tactics. Finally, recommendation tools examine past matches to discover which champion synergies are effective and which picks can counter the opponent, a capability directly useful to players during the selection phase in MOBA-style games. This requires clustering techniques to identify characteristic player styles or families of strategies.

Notably, in DotA 2, neural network and LSTM-based algorithms achieve real-time prediction accuracies of 85% to 90%. A landmark example is AlphaStar, developed by DeepMind for StarCraft II, which demonstrated that AI can excel in sophisticated strategy games: in 2019, the program reached Grandmaster level through multi-agent reinforcement learning, placing it among the top 0.2% of players worldwide. Nevertheless, challenges remain: models must adapt to regular game updates that alter the competitive environment, and providing interpretable explanations of their decisions is a key prerequisite for adoption by professionals. These advances motivate the development of more sophisticated methods for analysing the draft phase and optimising team compositions in League of Legends specifically.

2.1.2 Focus on League of Legends

League of Legends, developed by Riot Games, has established itself as one of the flagship titles of competitive esports, with over 150 million monthly players and a professional ecosystem generating millions of dollars in prize money. This popularity, combined with the availability of data through the official Riot Games API, makes it a privileged application domain for artificial intelligence research. Academic work on League of Legends has multiplied in recent years, exploring dimensions ranging from outcome prediction to team composition optimisation and the analysis of champion relationships. This section examines in detail the principal contributions in the literature, with emphasis on their methodologies, contributions, and limitations with respect to draft composition and champion synergies.

Early attempts to automate team composition in League of Legends relied primarily on classical optimisation methods. Oliveira et al. [1] propose a hybrid approach combining the MinMax algorithm with linear regression to construct balanced teams automatically. Their method rests on the assumption that each champion possesses measurable strengths and weaknesses, and that an optimal composition should maximise strengths while minimising vulnerabilities. The MinMax algorithm explores the possibility tree by anticipating the opponent's choices, while linear regression estimates a performance score for each configuration. Although this approach can generate potentially viable compositions, it suffers from a fundamental limitation: the absence of relational learning. Complex champion interactions, such as ability synergies or counter-strategies, are difficult to capture with predefined scores.

Costa et al. [2] propose an alternative based on genetic algorithms to optimise team composition. Inspired by natural evolution, a population of candidate compositions is generated randomly and then evolved across generations through selection, crossover, and mutation. The best-performing compositions according to a fitness function are preserved and recombined to yield potentially supe-

rior solutions. This approach efficiently explores a vast search space without requiring prior knowledge of optimal synergies. However, it demands significant computational resources and provides no explanation of why a given composition is judged favourable. Furthermore, defining the fitness function itself remains a challenge: how does one objectively quantify the quality of a composition before it has been tested in a real match?

A second family of work takes a more analytical perspective by mining match data to extract patterns and association rules. Ramler et al. [4] focus specifically on symbiotic relationships between champions, concentrating on ADC-Support duos in the bot lane. Analysing approximately 50,000 matches, they identify three types of ecological relationships: mutualism (both champions mutually benefit from each other's presence), commensalism (one champion benefits without affecting the other), and competition (both champions hinder each other). For instance, engage supports such as Leona create opportunities that AOE-heavy ADCs such as Miss Fortune can exploit effectively, illustrating a mutualistic relationship. While this analysis yields important insights into lane mechanics, it remains confined to champion pairs and does not account for the full five-champion team composition or interactions with the opposing team.

Cadman [7] proposes a more comprehensive analysis by combining several machine learning techniques: association rule mining via the Apriori algorithm, K-means clustering, and logistic regression. On a dataset of 96,000 matches collected through the Riot Games API, the study yields several noteworthy results. The Apriori algorithm identifies recurring champion associations: for instance, in low-Elo ranked games, picking Caitlyn frequently implies picking Lux, suggesting these champions are perceived as complementary or simply popular among less experienced players. K-means clustering with $K = 3$ or $K = 4$ highlights the interchangeability of certain champion classes, particularly Fighters and Slayers, which appear to have similar impacts on match outcomes. Logistic regression underlines the importance of the interaction between champion performance and player rank: certain champions such as Yuumi exhibit a very low win rate in Iron rank (37.9%) but become more effective at higher ranks, suggesting a steep learning curve. This multi-method approach provides a rich view of game dynamics, yet presents several limitations. Clustering requires fixing the number of clusters K arbitrarily, and the elbow method does not always reveal a clear optimum. The association rules generated by Apriori are primarily of the form "if champion A then champion B", capturing binary relationships without modelling the complex synergies that operate at the level of a full team. Finally, logistic regression achieves an AUC of approximately 0.54, indicating predictive capacity barely above chance.

Gilles [6] develops a related approach focused on teams from the League Championship Series (LCS) between 2018 and 2022. The study introduces metrics such as the Champion Mastery Value (CMV), Champion Performance Factor (CPF), and Overall Performance Rating (OPR) to quantify champion and player performance. Using Random Forest and logistic regression models, the author achieves high accuracy in win prediction. However, these metrics remain specific to the study's context and lack theoretical grounding, making them difficult to generalise to other datasets or to interpret intuitively.

Recommendation and deep learning approaches. A third category of work leverages recent advances in deep learning, and in particular graph neural networks, to model champion relationships. Horst et al. [3] propose DraftComPromise, a real-time recommendation system for the draft phase. The system relies on statistical analysis of match data to compute synergy scores between champions and win probabilities for different compositions, and presents users with an interface to simulate a draft and receive contextualised pick suggestions. While functional and practically usable, this system is based on descriptive statistics without explicit modelling of the underlying mechanisms.

Its recommendations lack interpretability: why is a given champion suggested, and which specific synergies are activated?

Chen et al. [5] take a further step by proposing MOBARec-GCNFP, a recommendation system based on a lightweight Graph Convolutional Network (GCN). Unlike previous approaches, this work explicitly models champions as graph nodes and their relationships as edges. The GCN learns vector representations (embeddings) for each champion by propagating information through the graph, thereby capturing synergies and counter-picks automatically. Trained on over 1.8 million matches from several MOBA games, the model demonstrates cross-game generalisation. However, MOBARec-GCNFP focuses exclusively on recommending the tenth and final champion, the last pick intended to counter the opposing team, and consequently neglects synergy with the four already-selected teammates, failing to account for the overall composition.

Finally, Costa et al. [8] conduct a detailed analysis of the features relevant to win prediction, focusing on data available after the picks-and-bans phase. Using Random Forest and logistic regression models, they achieve a remarkable AUC of 0.97, suggesting that team composition largely determines match outcome. Feature importance analysis reveals that certain champion combinations and counter-picks are particularly decisive. However, this predictive approach does not translate directly into actionable recommendations: knowing that a composition has a 70% chance of winning does not indicate which champion to add to improve those odds, nor why the composition is favourable.

2.2 Synthesis and Identified Limitations

The analysis of these works reveals a clear methodological progression, from classical optimisation approaches towards statistical and machine learning methods, and then towards graph-based deep learning. Nevertheless, several gaps persist. First, no existing approach integrates clustering to identify champion archetypes, graph theory to model relationships, and Graph Neural Networks for relational learning within a unified framework. Second, the majority of works analyse relationships in a binary or restricted manner, limited to champion pairs [4] or simple associations [7], without capturing the complexity of interactions within a full five-champion team. Third, explainability remains a major challenge: predictive models [6, 8] forecast outcomes but do not explain the underlying reasons, limiting their practical utility for coaches and players. Finally, clustering approaches such as that of Cadman [7] require the number of clusters to be fixed arbitrarily, with no objective method for determining the optimal partition.

These limitations motivate the development of a hybrid approach combining automatic community detection, multi-relational graph construction, and learning via Graph Attention Networks, the methodology developed in this work.

3 Theoretical Background

Analysing champions and their interactions requires handling a large number of variables, game statistics, win rates by matchup, and so on, while preserving the relational structure that makes the data meaningful. Addressing this problem requires appropriate mathematical tools. This section presents the theoretical foundations underpinning our methodology: graph theory, spectral analysis, and clustering.

3.1 Graph Theory

The study of relationships between discrete entities is a classical problem in mathematics. Graph theory provides the fundamental formal framework for modelling and analysing such relational

structures.

Definition 3.1 (Graph)**.** *A graph $G = (V, E)$ consists of a set of nodes (or vertices) V, representing here the champions, and a set of edges $E \subseteq V \times V$ connecting these nodes.*

Definition 3.2 (Weighted and directed graph)**.** *A graph is said to be* directed *if its edges are ordered pairs $(u, v) \in E$, allowing asymmetric relationships such as the counter-pick to be modelled. It is said to be* weighted *if a weight function $w : E \to \mathbb{R}$ associates a scalar value to each edge, for instance, the statistical advantage (gold lead) generated by a synergy.*

Definition 3.3 (Adjacency matrix)**.** *For a graph on n nodes, the adjacency matrix $A \in \mathbb{R}^{n \times n}$ is defined by*

$$A_{ij} = \begin{cases} w(v_i, v_j) & \text{if } (v_i, v_j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Definition 3.4 (Distance and shortest path)**.** *The distance $d(u, v)$ between two nodes is defined as the length of the shortest path connecting them. In a champion graph, a short distance between two vertices indicates strong semantic or strategic proximity.*

3.1.1 Centrality Measures

Centrality measures allow champion importance within the meta to be ranked hierarchically.

- **Node degree** (k_i): For an undirected graph, this is the number of neighbours of a vertex: $k_i = \sum_{j=1}^{n} A_{ij}$. A high degree characterises a versatile champion.
- **Closeness centrality**: Measures the efficiency of a node at diffusing information through the graph:
$$C(u) = \frac{n-1}{\sum_{v \neq u} d(u, v)}.$$
- **Betweenness centrality**: Identifies nodes acting as obligatory passage points between different strategies:
$$b(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}},$$
where σ_{st} is the total number of shortest paths from s to t and $\sigma_{st}(v)$ is the number of those paths passing through v.
- **Eigenvector centrality**: Captures the idea that the centrality of a champion depends on the centrality of its neighbours. It is defined by the eigenvalue equation $Ax = \lambda x$, where x is the vector of centrality scores.

3.1.2 Heterogeneous Graphs

The complexity of League of Legends cannot be captured solely by champion-to-champion relationships. Introducing different entity types, roles, items, objectives, motivates the use of heterogeneous graphs.

Definition 3.5 (Heterogeneous graph)**.** *A heterogeneous graph is a quintuple $G = (V, E, \mathcal{T}_V, \mathcal{T}_E, \phi, \psi)$, where $\mathcal{T}_V$ and $\mathcal{T}_E$ are the sets of node and edge types respectively, and $\phi : V \to \mathcal{T}_V$, $\psi : E \to \mathcal{T}_E$ are typing functions.*

In our modelling context, this structure accommodates multi-relational interactions. Formally, this translates into a multi-relational adjacency tensor $\mathcal{A} \in \mathbb{R}^{n \times n \times R}$, where each channel R encodes a specific relation type (synergy, counter-pick, role membership). This structure is particularly well suited to Graph Attention Network architectures, where attention weights can be learned differently for each relation type.

3.1.3 Community Detection

Community detection allows composition archetypes, such as Poke, Hard Engage, or Split-push, to be isolated in an unsupervised manner.

Definition 3.6 (Community)**.** *A community is a subset of vertices $C \subset V$ exhibiting a density of internal edges significantly higher than the density of edges towards the rest of the graph.*

Modularity. The relevance of a partition is evaluated using modularity, which measures the deviation between the number of internal edges in a community and the number expected under a null random graph model.

Definition 3.7 (Modularity)**.** *The modularity Q of a partition is defined by*

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j),$$

where m is the total number of edges in the graph, k_i and k_j are the respective degrees of nodes i and j, and $\delta(c_i, c_j)$ is the Kronecker delta, equal to 1 if i and j belong to the same community and 0 otherwise.

A high modularity (close to 1) indicates a strong community structure. Algorithms such as the Louvain method optimise this quantity iteratively, making them well suited to large champion interaction graphs.

3.2 Spectral Analysis

3.2.1 The Dimensionality Reduction Problem

Let a dataset consist of t observations $x_i \in \mathbb{R}^n$, where n is the (often high) initial dimension. Without loss of generality, assume the data are centred, i.e., $\sum_{i=1}^{t} x_i = 0$. The goal of dimensionality reduction is to find a representation of the data in a lower-dimensional space $\mathbb{R}^d$, with $d \ll n$, such that certain geometric or statistical properties are preserved. We seek a mapping $f : \mathbb{R}^n \to \mathbb{R}^d$ such that the projection retains as much information as possible according to a given criterion (variance, Euclidean distances).

3.2.2 Linear Dimensionality Reduction: Spectral Formulation

In the linear case, we seek a d-dimensional subspace spanned by an orthonormal family $\{u_1, \ldots, u_d\} \subset \mathbb{R}^n$. Let $X \in \mathbb{R}^{n \times t}$ be the data matrix, where each column corresponds to an observation x_i. The empirical covariance matrix is defined as

$$S = \frac{1}{t} X X^\top.$$

We seek projection directions that maximise projected variance. For a unit direction $w \in \mathbb{R}^n$, the variance of the projection $u = w^\top X$ is given by

$$\mathrm{var}(u) = w^\top S w.$$

The first principal axis solves the constrained optimisation problem

$$\max_w \; w^\top S w \quad \text{subject to} \quad w^\top w = 1.$$

Introducing a Lagrange multiplier λ_1, the stationarity condition of the Lagrangian $\mathcal{L}(w, \lambda_1) = w^\top S w - \lambda_1(w^\top w - 1)$ yields the eigenvalue equation

$$S w = \lambda_1 w.$$

Thus w is an eigenvector of S, and the maximum variance is attained at the largest eigenvalue λ_1. The d principal components are defined by the d eigenvectors associated with the d largest eigenvalues of S.

3.2.3 Variational Formulation and Low-Rank Approximation

Dimensionality reduction may also be interpreted as an approximation problem. Consider a rank-d linear model $f(y) = U_d y$, where $U_d \in \mathbb{R}^{n \times d}$ has orthonormal columns. The squared reconstruction error is

$$\mathcal{E} = \sum_{i=1}^{t} \|x_i - U_d U_d^\top x_i\|^2.$$

The solution minimising this error is given by the SVD of X: $X = U \Sigma V^\top$. The optimal U_d consists of the first d columns of U.

3.2.4 Spectral Analysis of Graphs

When the data (champions) are linked by a relational structure, the graph formalism becomes natural. Let $G = (V, E)$ be a weighted graph on n nodes with adjacency matrix $A \in \mathbb{R}^{n \times n}$. Define the degree matrix D by $D_{ii} = \sum_j A_{ij}$.

Definition 3.8 (Graph Laplacian)**.** *The unnormalised graph Laplacian is defined by $L = D - A$. This matrix is symmetric, positive semi-definite, and its eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$ characterise the connectivity of the graph.*

The eigenvectors associated with the smallest non-zero eigenvalues minimise the Dirichlet energy

$$\frac{1}{2} \sum_{i,j} A_{ij} (f_i - f_j)^2 = f^\top L f$$

subject to orthogonality constraints. This means that the components of an eigenvector f vary slowly between strongly connected nodes, a desirable property for representing champion relationships.

3.2.5 Spectral Dimensionality Reduction on Graphs

Spectral dimensionality reduction projects the nodes of the graph onto the d eigenvectors associated with the smallest non-zero eigenvalues of the Laplacian. Let $U_d \in \mathbb{R}^{n \times d}$ denote the matrix formed by these vectors. Each node i is then represented by a low-dimensional feature vector

$$z_i = (U_d)_{i,:}.$$

This projection guarantees that champions with strong relationships (frequent synergies or counter-picks) are mapped close to one another in $\mathbb{R}^d$.

3.2.6 Connection to Learned Representations

Modern methods such as Graph Neural Networks generalise these classical spectral approaches. Whereas classical methods rely on an explicit diagonalisation of L, graph convolutional layers learn filters that are polynomial functions of the Laplacian [10]:

$$H = \sigma\left(\sum_{k=0}^{K} \theta_k L^k X\right).$$

This connection motivates the use of GNN-based architectures as a principled extension of spectral graph analysis to the champion relationship modelling problem.

3.3 Clustering

Clustering is a fundamental step for structuring the League of Legends champion pool. The objective is to group champions according to their statistical similarities, burst damage, crowd control, mobility, tankiness, and so on, in order to identify coherent play profiles, often more precise than the official categories provided by Riot Games.

3.3.1 Classical Methods

K-means. K-means is one of the most widely used clustering algorithms, valued for its simplicity and efficiency. The central idea is to partition the data into K clusters based on proximity. The algorithm initialises K centroids randomly, then proceeds iteratively: at each step, it assigns every champion to its nearest centroid, then recomputes each centroid as the mean of the features of its assigned champions. Although highly efficient on large datasets, K-means requires the number of groups K to be specified in advance, typically via the elbow method, and struggles to identify clusters of irregular shape or widely differing densities.

Hierarchical clustering. Unlike K-means, which produces a fixed partition, hierarchical clustering constructs a hierarchy of groups represented as a tree called a *dendrogram*. Two strategies exist: agglomerative (starting from each champion as an isolated group and progressively merging them) and divisive (starting from a single group and successively splitting it). The choice of linkage criterion determines which groups are merged at each step:

- *Single linkage*: the distance between the two closest champions across groups.
- *Complete linkage*: the distance between the two most distant champions across groups.

- *Ward linkage*: groups are merged to minimise the increase in internal variance, the most commonly used criterion, as it tends to produce balanced-size clusters.

The dendrogram allows champion proximity to be visualised at multiple scales: for instance, all mages may cluster together before subdividing into control mages (Azir, Syndra) and battle mages (Ryze, Kassadin).

DBSCAN. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) defines a cluster as a region of high point density. Champions in isolated low-density regions are treated as noise, or outliers. Its key parameters are ε, the maximum distance for two champions to be considered neighbours, and MinPts, the minimum number of champions required within radius ε to form a cluster. Crucially, DBSCAN requires no prior specification of the number of clusters. It can discover clusters of complex shapes and, importantly, identify off-meta picks that do not fit any established category in the current meta, a property directly relevant to the draft context.

4 Methodology

This section details the full pipeline implemented to model champion relationships and predict draft outcomes. The approach proceeds in five stages: data collection and preprocessing, champion representation via style vectors, synergy modelling through collaborative filtering embeddings, feature engineering, and supervised classification.

4.1 Data Collection and Preprocessing

Match data are collected from the Master+ ranked ladder on the EUW server using the Riot Games API, covering patch 26.S1.3. The raw dataset is structured at the player level, with one row per player per game, recording champion selection, individual statistics, and match outcome. Only games played in CLASSIC mode are retained, and matches are filtered to include only players assigned to team identifiers 100 or 200. Champion names are normalised to remove extraneous whitespace.

The player-level data are then aggregated into a game-level representation. For each match, the five champions selected by each team are grouped into ordered lists `team100_champs` and `team200_champs`, yielding one row per game with a binary outcome variable `win` encoding whether team 100 won. Matches for which either team does not contain exactly five players are discarded. This aggregation produces two parallel data structures: `df_raw`, retained at player level for the construction of champion style vectors, and `games_df`, used at game level for all subsequent modelling steps. The dataset is split into training and test sets with an 80/20 ratio (`random_state` = 42). A minimum threshold of 20 games is applied when computing pairwise statistics, ensuring reliability of the estimated win rates.

4.2 Champion Representation via Style Vectors

Each champion is characterised by two complementary style vectors, constructed from aggregated match statistics over all games in which that champion appears. A minimum of 200 appearances is required for a champion to receive a style vector, ensuring statistical robustness.

4.2.1 Macro Style Vectors

Macro style vectors are built from team-level statistics and capture a champion's strategic contribution to the team as a whole. Seven dimensions are defined:

- **frontline**: damage taken per minute, measuring the champion's capacity to absorb pressure for the team.
- **dps**: total damage dealt to champions per minute.
- **cc_density**: crowd control time inflicted per minute, reflecting zone-control contribution.
- **vision_per_min**: vision score per minute, encoding map control and information gathering.
- **obj_control**: a composite score aggregating dragon kills, baron kills (weighted $\times 3$), turret kills, and inhibitor kills per minute, capturing objective-focused playstyles.
- **teamfight**: damage dealt to objectives per minute, as a proxy for teamfight participation around major objectives.
- **early_pressure**: the inverse of average game duration, distinguishing snowball-oriented compositions from late-game scaling ones.

All dimensions are normalised per minute for consistency across games of varying duration. The resulting macro style vector for champion c is a vector $\mathbf{m}_c \in \mathbb{R}^7$.

4.2.2 Micro Style Vectors

Micro style vectors are constructed from individual player statistics and capture each champion's personal gameplay profile. Ten dimensions are defined:

- **kda**: effective kill participation, combining kills, assists, and deaths into a single aggression/survival signal.
- **cs_per_min**: creep score per minute, encoding farm dependency and distinguishing carry from roaming roles.
- **gold_per_min**: gold income per minute, reflecting the champion's economic weight within the composition.
- **dmg_share**: proportion of the team's total damage dealt by this champion, identifying primary damage carries.
- **tank_share**: proportion of the team's total damage received, measuring actual frontlining behaviour.
- **magic_ratio**: proportion of damage dealt that is magic, capturing damage-type diversity.
- **utility_per_min**: healing and shielding applied to allies per minute, identifying enchanters and active supports.
- **vision_per_min**: individual vision score per minute.
- **cc_per_min**: individual crowd control time inflicted per minute.

- **early_events**: kill and turret participation rate in the early game, distinguishing early-pressure from scaling profiles.

No role-based normalisation is applied: the role signal is already implicitly encoded in the per-champion averages, since a champion such as Malphite will naturally exhibit high **tank_share** while a champion such as Jinx will exhibit high **cs_per_min**. The resulting micro style vector for champion c is a vector $\boldsymbol{\mu}_c \in \mathbb{R}^{10}$.

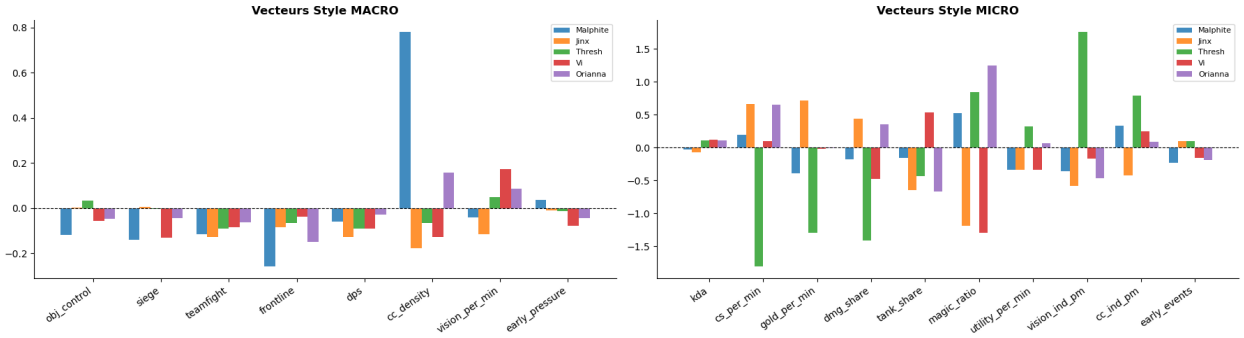


Figure 1: Macro (left) and micro (right) style vectors for five representative champions: Malphite (tank/engage), Jinx (ADC carry), Thresh (support), Vi (jungle), and Orianna (mid mage). The contrasts illustrate how each vector captures distinct strategic roles without explicit role labelling.

4.3 Synergy Modelling via Collaborative Filtering

To capture champion synergies beyond raw co-occurrence statistics, we train two LightFM embedding models [13] on the same interaction matrix, following an uplift-based approach.

4.3.1 Interaction Matrix Construction

Each game contributes two interaction records, one per team. For each team–champion pair, a base interaction of weight 1.0 is recorded in the *popularity* matrix, and a win-weighted interaction of weight $1.0 + \alpha \cdot w$ is recorded in the *win* matrix, where $w \in \{0, 1\}$ is the team's outcome and $\alpha = 1.0$ controls the strength of the win signal. Both matrices are stored as sparse COO matrices of shape $(2 \cdot |\text{games}|) \times |\text{champions}|$.

4.3.2 Embedding Models

Two models are trained independently on these matrices with $d = 32$ latent components and $T = 30$ epochs:

- **model_pop**: trained on the base interaction matrix, capturing champion co-occurrence popularity.
- **model_win**: trained on the win-weighted matrix, capturing genuine win-predictive synergies.

Let $\mathbf{e}_c^{\text{win}} \in \mathbb{R}^{32}$ and $\mathbf{e}_c^{\text{pop}} \in \mathbb{R}^{32}$ denote the embeddings of champion c from each model respectively.

4.3.3 Uplift Score

The pairwise synergy signal between two champions a and b is decomposed into three quantities:

$$\text{fm\_synergy}(a, b) = \mathbf{e}_a^{\text{win}} \cdot \mathbf{e}_b^{\text{win}}, \tag{1}$$

$$\text{uplift}(a, b) = \mathbf{e}_a^{\text{win}} \cdot \mathbf{e}_b^{\text{win}} \; - \; \mathbf{e}_a^{\text{pop}} \cdot \mathbf{e}_b^{\text{pop}}. \tag{2}$$

The uplift score eliminates the popularity bias inherent to co-occurrence statistics: a champion pair with high `fm_synergy` but low `uplift` is frequently picked together primarily because both champions are meta-dominant, whereas a pair with high `uplift` exhibits a win-rate advantage that cannot be explained by popularity alone.

4.4 Feature Engineering

The `DraftFeatureEngineering` class aggregates the signals described above into a fixed-length feature vector for each match. Four synergy signal types are combined, as summarised in Table 1.

Table 1: Synergy signals integrated into the feature vector.

Signal	Source	Description
`delta_wr`	Raw statistics	Duo win rate minus mean individual win rate
`fm_synergy`	LightFM win embedding	$\mathbf{e}_a^{\text{win}} \cdot \mathbf{e}_b^{\text{win}}$
`uplift`	LightFM win − pop	Net synergy, popularity-corrected
`macro_comp`	Style vectors	Tactical complementarity between $\mathbf{m}_a$ and $\mathbf{m}_b$

For each team, all $\binom{5}{2} = 10$ champion pairs are evaluated on each of these signals. The resulting per-team values are summarised into scalar aggregates, mean, minimum, and maximum, yielding a compact representation. The *advantage* features are computed as the difference between the blue-side and red-side aggregates, encoding the relative strength of each team's composition. In addition, the 25 cross-team counter-pick win rates are computed, along with the individual win rates of the five champions on each side. Champion presence indicators are included as binary features. The final feature vector concatenates all scalar aggregates, raw pairwise values, and presence indicators.

4.5 Classification Models

The win prediction task is framed as binary classification on the feature vector described above. Nine classifiers are trained and evaluated: Logistic Regression, Random Forest (200 trees, minimum leaf size 5), XGBoost, LightGBM, CatBoost, a Support Vector Machine, K-Nearest Neighbours, a Multilayer Perceptron, and TabNet. In addition, a stacking ensemble and a soft-voting ensemble are constructed from subsets of these base learners. All models are evaluated on the held-out test set using the area under the ROC curve (AUC-ROC) as the primary metric. The best single model is identified by this criterion and used for all subsequent analyses and predictions.

5 Results

5.1 Dataset and Experimental Setup

The soloQ dataset consists of 16,402 ranked games after filtering, covering 171 unique champions. Katarina was removed from the dataset prior to training: her exclusion improves signal purity by

approximately 0.008 AUC on the validation set, suggesting that her presence introduces noise that is not captured by the draft features alone. The dataset is split into 13,121 training games and 3,281 test games (80/20, `random_state` = 42). The professional dataset consists of 979 games from the 2026 LCK, LEC, and LCS seasons, used exclusively for out-of-distribution evaluation and never seen during training.

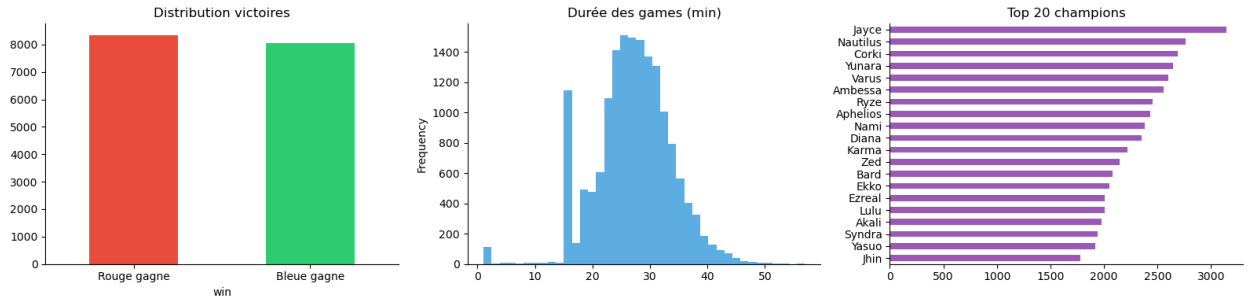


Figure 2: Dataset overview: win/loss balance (left), game duration distribution in minutes (centre), and top 20 most-played champions by frequency (right).

5.2 Champion Filtering: Empirical Identification of Outlier Champions

Removing Katarina from the dataset prior to training increases AUC from 0.5629 to 0.5714 (+0.008), an improvement that exceeds the gains obtained by any of the four feature engineering configurations tested. Katarina represents approximately 1,000 games in the dataset (∼5% of the total), a sufficient sample for the effect to be considered statistically meaningful rather than incidental. We interpret this result as evidence that certain champions may introduce a confounding signal in the draft-to-outcome relationship: for such champions, the match outcome may be more strongly correlated with individual player performance than with the composition of the team, making their presence uninformative or misleading for draft modelling purposes.

This observation motivated an investigation into whether such champions could be identified automatically. Two approaches were evaluated. The first computed, for each champion, the win rate gap between games in the top and bottom KDA terciles:

$$s_c = \mathrm{WR}(c,\ \mathrm{KDA} > P_{67}) - \mathrm{WR}(c,\ \mathrm{KDA} < P_{33})$$

under the hypothesis that a champion whose outcome is primarily driven by individual performance would exhibit an anomalously high sensitivity to KDA. The approach failed: the mean sensitivity across all champions was 0.77 ± 0.07, and Katarina's Z-score was 0.844, well within the noise. This structural outcome reflects the fact that in any team game, individual performance is correlated with victory for every champion, regardless of role or play style. A second approach combined win rate variance over sliding windows, individual-to-team win rate correlation, and Sarle's bimodality coefficient. This produced inconsistent results: Braum, Taric, and Kindred, all pro-relevant utility champions, were ranked among the highest-scoring candidates, confirming that the metrics were not capturing the intended signal.

Given the failure of automated approaches, we adopted an empirical strategy: manually testing candidates identified on the basis of game knowledge as having atypical soloQ profiles (Master Yi, Malzahar, Shaco, among others), and retaining only those whose removal yields a measurable improvement in AUC on the validation set. Among the candidates tested, only Katarina produced a consistent and positive effect; the others either degraded performance or had no measurable impact.

The underlying difficulty, separating draft-level effects from individual skill effects using binary outcome data, remains an open problem and a direction for future work.

5.3 Exploratory Data Analysis

5.3.1 Hypothesis Testing

Before training any predictive model, we conduct five statistical hypothesis tests to verify that the draft phase carries a detectable signal and to characterise its nature.

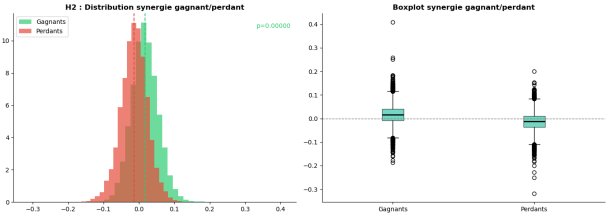
H1, Healing champions do not confer a win rate advantage. Teams containing at least one healing champion win at a rate of 49.9% versus 50.1% for teams without ($z = -0.53$, $p = 0.596$, not significant). The mere presence of a healer is not a reliable draft signal in isolation.

H2, Compositions with stronger internal synergies win more often. The mean intra-team synergy score (delta win rate across all $\binom{5}{2} = 10$ champion pairs) is +0.017 for winning teams versus −0.013 for losing teams (Mann-Whitney, $p < 10^{-6}$). Pairwise synergy carries a statistically significant predictive signal.

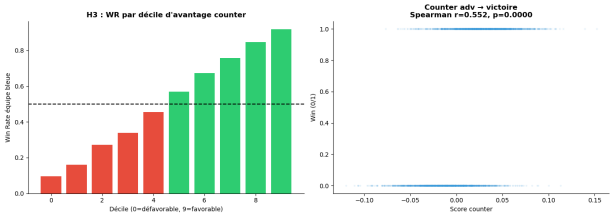
H3, Counter-pick advantage correlates with victory. The mean cross-team counter advantage is +0.019 for winners versus −0.001 for losers (Mann-Whitney, $p < 10^{-6}$). Historical matchup win rates provide a complementary and independent signal from internal synergies.

H4, Engage and poke compositions do not differ in game duration. Engage compositions last on average 27.1 minutes versus 27.4 minutes for poke compositions (Mann-Whitney, $p = 0.299$, not significant). Composition archetype alone does not determine game tempo at this rank.

H5, Blue side holds a statistically significant advantage. Blue side wins 50.9% of games ($z = 3.14$, $p = 0.002$, 95% CI: $[+0.006, +0.028]$), consistent with the first-pick advantage documented in the competitive literature.



(a) H2 – Synergy score distribution: winning vs. losing teams.



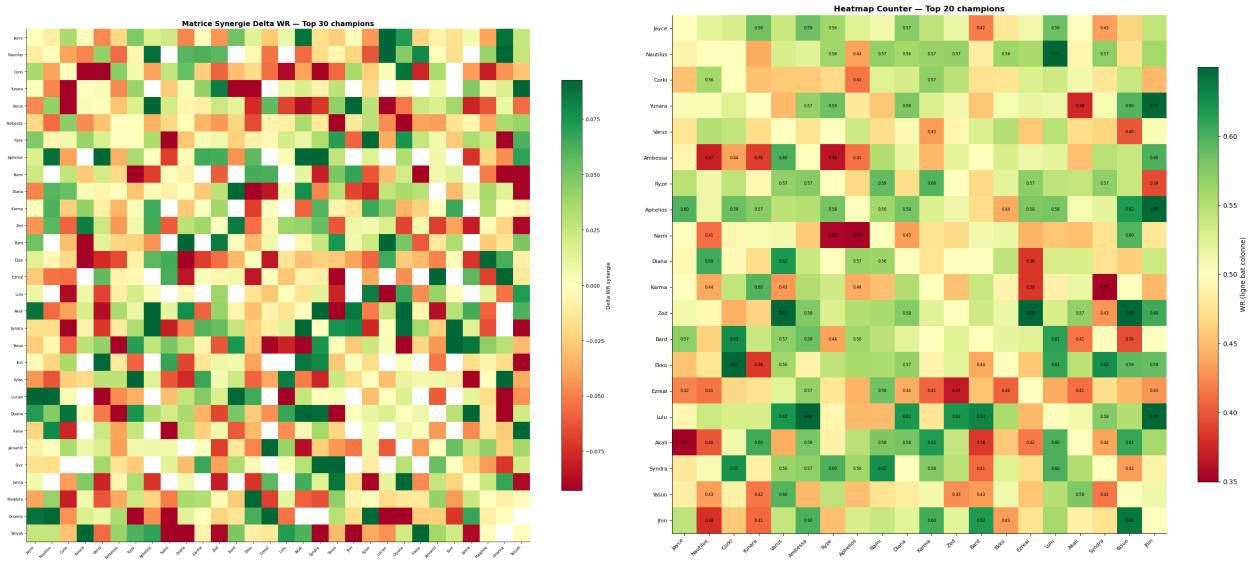
(b) H3 – Counter-pick advantage distribution: winning vs. losing teams.

Figure 3: Statistically significant hypothesis tests (Mann-Whitney, $p < 10^{-6}$ for both H2 and H3). The distributions confirm that pairwise synergy and counter-pick advantage carry independent predictive signals.

5.3.2 Synergy and Counter-Pick Matrices

Computing delta win rates across all observed champion pairs reveals structured patterns in the synergy space. Among the top positive synergies with sufficient sample size ($n \geq 15$), Kai'Sa–Taliyah (ΔWR $= +0.189$, $n = 71$) and Syndra–Sivir (ΔWR $= +0.154$, $n = 74$) stand out as the strongest in our dataset. Among negative synergies, Nami–Kai'Sa (ΔWR $= -0.197$, $n = 39$) indicates compositions that lose more often than their individual win rates would predict. The

counter-pick matrix on the 20 most-picked champions reveals that asymmetries rarely exceed ±10% win rate, confirming that counter-picks provide a real but moderate signal.



(a) Delta win rate synergy matrix (top 30 champions). Green cells indicate positive synergy, red cells negative synergy.

(b) Counter-pick win rate matrix (top 20 champions). Cell (i, j) encodes the win rate of champion i when facing champion j.

Figure 4: Synergy and counter-pick matrices computed from the training set. Most asymmetries remain within ±10% win rate, confirming that individual matchup advantages are real but moderate.

5.3.3 Champion Clustering

K-means clustering applied to the macro style vectors identifies $K = 4$ archetypes, selected by the elbow criterion (Figure 9). None of the four clusters exhibits a statistically significant win rate deviation from 50% (all within ±0.5%), confirming that archetype membership is a structural property of the champion pool rather than a predictive feature in isolation. PCA on the macro style vectors and subsequent t-SNE projection reveal a continuous latent space: champions such as Malphite (high `cc_density`, high `frontline`) and Jinx (high `cs_per_min`, high `gold_per_min`) occupy well-separated regions, while multi-role champions occupy intermediate positions consistent with their strategic versatility.

5.3.4 Functional Coverage Analysis

Applying Mann-Whitney tests on the functional coverage scores (sum of positive contributions per tactical axis across the five champions of a team) shows that six of eight axes discriminate significantly between winning and losing compositions: objective control, siege, teamfight, frontline, DPS, and vision ($p < 0.01$ for all six). Crowd control density and early pressure do not reach significance ($p = 0.658$ and $p = 0.744$ respectively). This finding motivates the inclusion of team-level functional coverage as a feature in versions V3 and V4, as a complement to the pairwise synergy signal.

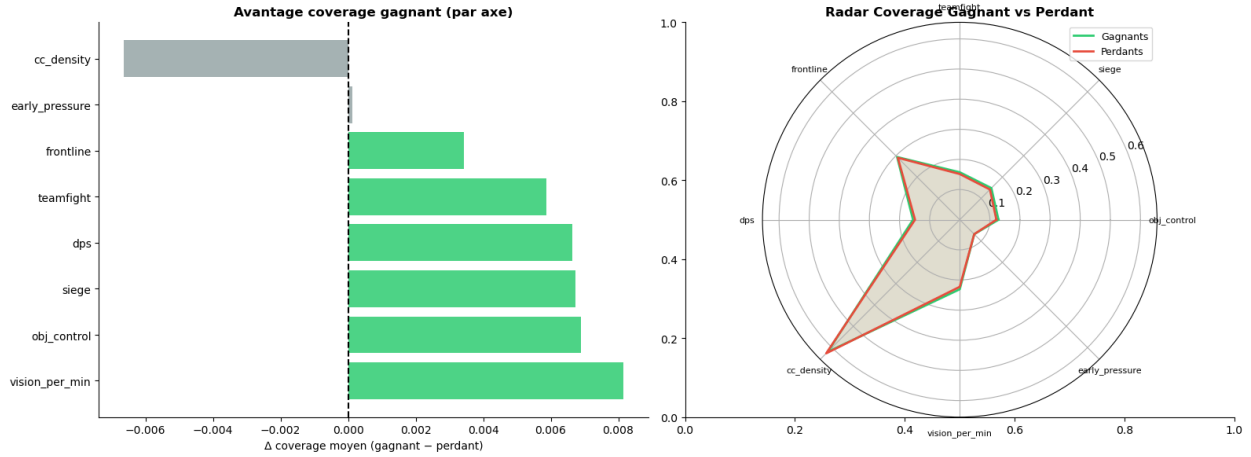


Figure 5: Functional coverage analysis. Left: mean coverage advantage (winning minus losing teams) per tactical axis. Six of eight axes are statistically significant ($p < 0.01$); crowd control density and early pressure are not. Right: radar comparison of winning versus losing team coverage profiles.

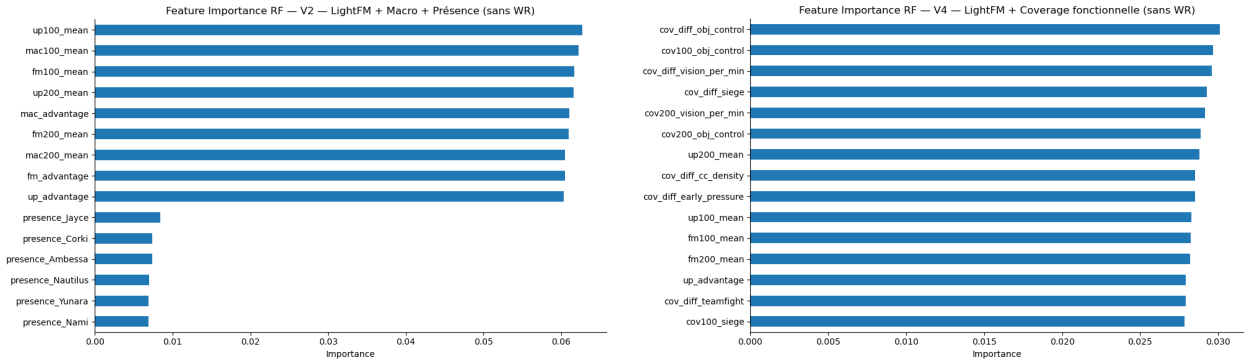
5.4 Comparative Evaluation of Feature Sets

Four feature engineering configurations are evaluated across nine classifiers plus two ensemble variants. Table 2 summarises the best AUC-ROC achieved per version on the held-out test set.

Table 2: Best AUC-ROC per feature version on the soloQ test set ($n = 3{,}281$).

Version	**Features**	**Best model**	**AUC**
V1	WR + LightFM + Macro + Presence	LDA / Ridge / LR	0.5699
V2	LightFM + Macro + Presence (no WR)	Ridge / LDA	0.5704
V3	WR + LightFM + Coverage + Presence	LDA / LR	0.5705
V4	LightFM + Coverage + Presence (no WR)	Ridge / LDA	**0.5714**

The best overall result is AUC = 0.5714, achieved jointly by Ridge and LDA on version V4. A consistent pattern emerges across all four versions: linear models (LDA, Logistic Regression, Ridge, Lasso, ElasticNet) systematically outperform tree-based models (Random Forest, MLP) and the graph-based model. The performance gap is most pronounced in V4: Random Forest achieves AUC = 0.5125 while Ridge reaches 0.5714 on the same feature set. Among the remaining classifiers, TabNet performs comparably to tree-based models but remains below the linear baselines, likely constrained by the limited dataset size. The Support Vector Machine and K-Nearest Neighbours consistently underperform relative to linear models, with KNN particularly sensitive to the high dimensionality of the presence indicator features. A full model comparison on the initial feature set, prior to the V1–V4 improvements, is provided in Table 5 in Appendix C. That preliminary benchmark motivated the exclusion of several architectures from the final evaluation: XGBoost DART and TabNet were discarded due to prohibitive training times (412s and 488s respectively) with no corresponding accuracy gain, and the degree-2 polynomial Logistic Regression was found to be counterproductive (AUC = 0.5158).



(a) Feature importance – V1 (WR + LightFM + Macro + Presence). Win rate advantage (`wr_advantage`) dominates by a wide margin.

(b) Feature importance – V4 (LightFM + Coverage, no WR). Without win rates, coverage and uplift features contribute more evenly.

Figure 6: Random Forest feature importance for versions V1 and V4. The contrast illustrates how removing individual win rates forces the model to exploit genuinely relational signals (coverage, LightFM uplift).

5.5 Graph Attention Network

The DraftGATv2 model is trained on 8,554 graph-structured instances, with each draft represented as a ten-node graph (five champions per team), node features of dimension 44 (win rate, LightFM embeddings, functional coverage, team membership flag), and edges encoding intra-team synergies and cross-team counter-picks. The architecture uses two GATv2Conv layers with batch normalisation and separate pooling per team. The model converges at epoch 32 (early stopping, patience = 20), with training loss decreasing from 0.685 to 0.604. It achieves AUC = 0.5465 on the test set, below all linear baselines. Figure 7 illustrates a concrete prediction output, showing how the ensemble combines win probability, model consensus, pairwise Δwr synergies, and LightFM uplift scores into an interpretable breakdown for a given draft.

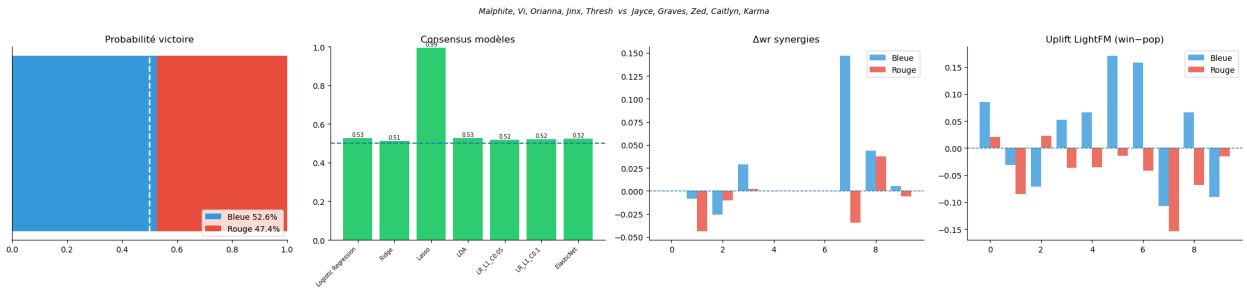


Figure 7: Prediction output for Malphite/Vi/Orianna/Jinx/Thresh (blue) vs Jayce/Graves/Zed/Caitlyn/Karma (red). From left to right: ensemble win probability (52.6% blue), per-model consensus, intra-team Δwr synergies per pair, and LightFM uplift scores. The four panels together provide an interpretable breakdown of why the model favours the blue composition.

5.6 Professional Data Integration Experiments

Since the ultimate goal of the pipeline is to inform draft decisions in a professional context, training exclusively on soloQ data raises a question of ecological validity. We therefore investigated four

strategies for incorporating the 979 Oracle's Elixir professional games into the training set, prior to adopting the out-of-distribution evaluation reported in Section 5.7.

Preparing the professional data required aligning two structurally different formats: position labels, side encoding, and missing statistics. A champion name normalisation issue was also identified, three champions had different spellings across the two datasets (KaiSa, KhaZix, LeBlanc), artificially inflating the unique champion count from 171 to 174 before correction.

Experiment 1, Direct merge. The 979 professional games are concatenated to the ∼17,000 soloQ games and the models retrained. AUC = 0.5642, a decrease of 0.007 relative to the soloQ-only baseline. The professional games (∼5% of the merged dataset) introduce compositional patterns that differ substantially from soloQ, coordinated team compositions, identity picks, absence of individual carry dynamics, and lack sufficient mass to override the soloQ signal.

Experiment 2, Weighted merge. A sample weight of 5.0 is assigned to professional games versus 1.0 for soloQ games via the `sample_weight` parameter of the scikit-learn estimators. AUC = 0.5636, a marginal improvement over Experiment 1 but still below the soloQ baseline. The limited effect is consistent with the professional signal remaining structurally different from the soloQ signal regardless of its weight.

Experiment 3, Pro-relevant champion filter. Rather than enriching the dataset, this approach restricts soloQ games to those in which all ten champions belong to the professional champion pool (153 of 171 champions). AUC = 0.5513. The filter removes approximately 60% of the soloQ dataset, leaving insufficient data for reliable learning.

Experiment 4, Filter and weighting combined. Applying both the pro-relevant filter and the 5.0 sample weight yields AUC = 0.5513, identical to Experiment 3. The additional weighting provides no benefit when the filtered dataset is already too small.

Table 3 summarises these results. None of the four integration strategies improves upon the soloQ-only baseline. This outcome reflects the fundamental distributional shift between the two contexts: soloQ compositions are guided by individual win rates and meta picks, while professional compositions are shaped by team strategy, targeted counter-picks, and identity picks specific to each organisation. These two distributions are sufficiently different that naïve mixing degrades rather than improves predictive performance with the available data volumes.

Table 3: AUC-ROC of the best linear model under four professional data integration strategies, compared to the soloQ-only baseline.

Strategy	**Best AUC**
SoloQ only (baseline)	**0.5714**
Experiment 1, Direct merge	0.5642
Experiment 2, Weighted merge	0.5636
Experiment 3, Pro-relevant filter	0.5513
Experiment 4, Filter + weighting	0.5513

These results motivate the decision to train on soloQ only and evaluate generalisation to professional play as an external validation, reported in the following section.

5.7 Out-of-Distribution Evaluation on Professional Data

The models trained exclusively on soloQ data are applied without retraining to 979 professional games from the 2026 LCK, LEC, and LCS seasons. The best generalisation is achieved by the

V2 configuration (LightFM + Macro, no win rate) with Logistic Regression L1 ($C = 0.1$): AUC = 0.5525, accuracy = 53.7%. Table 4 summarises the top results on professional data.

Table 4: Generalisation to 979 professional games (out-of-distribution evaluation).

Version	Model	AUC soloQ	AUC pro	Acc pro
V2	LR L1 $C = 0.1$	0.5701	**0.5525**	**53.7%**
V2	ElasticNet	0.5703	0.5510	53.4%
V2	LR	0.5703	0.5486	53.3%
V4	LDA	0.5714	0.5468	53.5%
V4	LR	0.5713	0.5502	53.3%

Accuracy increases monotonically with prediction confidence: games predicted with a 20–30% margin are correctly classified 58.7% of the time, and those with a 30–40% margin 60.0%. Figure 8 shows the ROC curve, predicted probability distribution, and calibration curve for the best model. The calibration curve confirms that the model is well calibrated in the 0.45–0.65 range but slightly underconfident at the extremes, consistent with a weak and diffuse compositional signal.

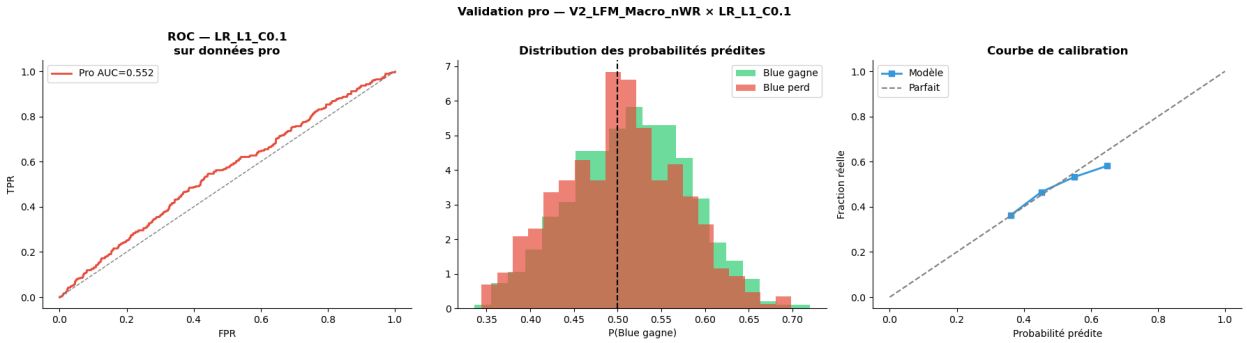


Figure 8: Professional data validation for the best model (V2, LR L1 $C = 0.1$, AUC = 0.552). Left: ROC curve on 979 professional games. Centre: distribution of predicted blue-side win probabilities by actual outcome. Right: calibration curve showing the model is well calibrated in the 0.45–0.65 range but slightly underconfident at the extremes.

6 Discussion

6.1 The Draft Signal is Real but Weak and Linear

The most consistent finding across all experiments is the systematic superiority of linear models over non-linear alternatives. This holds across all four feature versions and all classifier families. One possible interpretation is that the relationship between draft composition and match outcome in high-Elo soloQ is predominantly additive: the presence or absence of each champion may contribute independently to the win probability, without strong interactive effects requiring non-linear modelling. This is consistent with prior work by Costa et al. [8], who report near-linear predictive relationships in post-draft features.

6.2 Individual Win Rates Introduce Noise

A counter-intuitive but robust finding is that removing individual champion win rates from the feature set consistently improves performance for linear models (V4 vs. V3, V2 vs. V1). Individual soloQ win rates may reflect the current meta and player-pool biases rather than the intrinsic contribution of a champion to a specific composition. Removing this signal forces the model to rely on genuinely relational features, LightFM embeddings, coverage, and presence indicators , producing a purer compositional signal. This has a direct practical implication: draft recommendation tools should not rank champions by their individual win rates alone.

6.3 The Uplift Score Isolates Genuine Synergies

The LightFM uplift score (win embedding dot product minus popularity embedding dot product) proves more informative than raw co-occurrence statistics. By subtracting the popularity bias, it separates champion pairs that win together because they are individually strong in the meta from pairs that may exhibit a genuine compositional advantage. Among the highest-scoring pairs in the discovery score analysis, Orianna–Varus and Kai'Sa–Taliyah stand out, which may suggest that these combinations carry synergistic value beyond their individual meta presence.

This correction is empirically justified by a statistically significant positive correlation between individual champion win rate and pick frequency (Spearman $r = 0.245$, $p = 0.0012$, Figure 11 in Appendix). Champions played more often tend to have a slightly higher win rate, reflecting both the current meta favouring certain picks and the selection bias of experienced players gravitating towards effective champions. Without correcting for this popularity signal, co-occurrence-based synergy scores conflate genuine compositional advantage with the effect of two individually strong champions being simultaneously present in the meta. The uplift score addresses this directly by design.

6.4 Why the GNN Underperforms

The DraftGATv2 achieves AUC = 0.5465, below all linear baselines. This underperformance is consistent with the dataset size: with 13,121 training instances and approximately 14,535 possible champion pairs, most pairs appear too infrequently for the attention mechanism to learn specialised edge weights. The training loss converges correctly but AUC plateaus, a pattern commonly observed in graph-structured models trained on sparse data. More generally, graph attention networks have been shown to require substantially larger datasets to outperform linear baselines in settings with sparse entity-pair interactions [10], and our results are consistent with this observation.

6.5 Partial Generalisation to Professional Play

The observation that a model trained exclusively on soloQ Master+ achieves AUC = 0.5525 and accuracy = 53.7% on professional games is the most practically significant result of this study. It demonstrates that compositional patterns learned from high-Elo ranked play transfer partially to the professional context without domain adaptation. The version that generalises best (V2, no win rates) differs from the version that performs best on soloQ (V4), reinforcing the interpretation that soloQ win rates are contextually misleading: they reflect ranked ladder dynamics that do not hold in a coordinated professional environment. The accuracy improvement with prediction confidence (58.7% at 20–30% margin, 60.0% at 30–40% margin) suggests that the model's confidence is directionally calibrated on professional data.

6.6 Limitations

Several limitations must be acknowledged. First, the model covers only the high-Elo soloQ context and cannot be applied directly to other ranks or as a standalone professional draft tool without domain adaptation. Second, draft order information is absent from the current feature representation despite being strategically decisive in professional play: which team picks first, targeted bans, and positional priority are all ignored in the current formulation. Third, the professional validation set of 979 games covers only two patches, which is insufficient for statistically robust conclusions. Fourth, the patch update cycle of every two weeks means that statistics and embeddings must be recomputed periodically to remain valid, limiting the model's shelf life without continuous data collection. Fifth, the current approach restricts each model to a single patch to ensure distributional consistency; extending the dataset to cover a full season across multiple patches would substantially increase the number of training examples, but would require identifying and excluding champions that underwent significant reworks or balance changes between patches, as their statistical profiles would be incommensurable across versions. Sixth, the model treats all occurrences of a champion uniformly, regardless of the player controlling it. In practice, a champion played by a specialist, a one-trick player with hundreds of games on that champion, carries a systematically different win rate than the same champion played by a generalist. Incorporating player-level proficiency as a contextual feature, for instance through a champion mastery score or an encounter-specific performance prior, could improve both predictive accuracy and the interpretability of composition assessments.

7 Conclusion

This paper presented a hybrid pipeline for modelling champion relationships and predicting match outcomes from the draft phase in League of Legends, combining champion style vectors, LightFM collaborative filtering embeddings, functional coverage analysis, and a comparative evaluation across nine classifiers and a Graph Attention Network. The pipeline was evaluated on 16,402 high-Elo soloQ games and validated externally on 979 professional games from the 2026 LCK, LEC, and LCS seasons.

The principal empirical findings are threefold. First, linear models systematically outperform non-linear alternatives across all four feature configurations tested, suggesting that the draft-to-outcome relationship in high-Elo soloQ is predominantly additive. Second, removing individual champion win rates from the feature set consistently improves performance, indicating that soloQ win rates may reflect meta and selection biases rather than genuine compositional value. Third, the uplift score, which corrects for the statistically significant correlation between champion popularity and win rate (Spearman $r = 0.245$, $p = 0.0012$), proves more discriminative than raw co-occurrence synergy scores. The best model achieves AUC $= 0.5714$ on the soloQ test set and generalises to professional play with AUC $= 0.5525$ and accuracy $= 53.7\%$ without retraining, demonstrating that compositional patterns learned from ranked data carry partial predictive value in a structurally different competitive context.

Several directions remain open. Extending the dataset across multiple patches while controlling for champion reworks would substantially increase training volume. Incorporating player-level proficiency, such as champion mastery scores, could improve both accuracy and interpretability by accounting for the performance gap between specialists and generalists. Encoding draft order information would better reflect the sequential nature of the pick-and-ban phase. Finally, scaling the dataset to the order of magnitude required for the Graph Attention Network to outperform linear baselines would allow the full relational modelling potential of the architecture to be assessed.

References

[1] V. da Costa Oliveira, B. J. Placides, M. de Freitas Oliveira Baffa, and A. F. da Veiga Machado, "A Hybrid Approach To Build Automatic Team Composition In League of Legends," Instituto Federal de Educação, Ciência e Tecnologia do Sudeste de Minas Gerais, Brasil.

[2] L. M. Costa, A. C. Corrêa Souza, and F. C. M. Souza, "An Approach for Team Composition in League of Legends using Genetic Algorithm," Software Engineering Coordination, University of Technology-Paraná, Dois Vizinhos, Brazil.

[3] R. Horst, F. Meyer, and R. Dörner, "DraftComPromise – On Draft Composition Recommendations in League of Legends," RheinMain University of Applied Sciences, Wiesbaden, Germany.

[4] I. Ramler, C.-S. Lee, and M. Schuckers, "Identifying Symbiotic Relationships between Champions in League of Legends," Department of Mathematics, Computer Science, and Statistics, St. Lawrence University, Canton, NY.

[5] C. Chen, C. Bai, F. Mo, X. Fan, and H. Yamana, "MOBARec-GCNFP: Champion Recommendation for Multi-Player Online Battle Arena Games Using Graph Convolution Network with Fewer Parameters," Waseda University, Tokyo, Japan.

[6] A. L. Gilles, "Statistical Analysis and Machine Learning to Improve League Championship Series Teams," M.S. thesis, Dept. Information Systems Technology, California State University, San Bernardino, CA, 2023.

[7] A. Cadman, "Studying the Effects of Team Composition in Role-based Competitive Video Games," MSci Hons thesis, Mathematics with Statistics, Lancaster University, May 2024.

[8] L. M. Costa, R. G. Mantovani, F. C. M. Souza, and G. Xexéo, "Feature Analysis to League of Legends Victory Prediction on the Picks and Bans Phase," Federal University of Rio de Janeiro (UFRJ), Rio de Janeiro, Brazil.

[9] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[10] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, "Simple and Deep Graph Convolutional Networks," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, vol. 119, Proceedings of Machine Learning Research, 2020, pp. 1725–1735.

[11] A. Ghodsi, "Dimensionality Reduction: A Short Tutorial," Dept. of Statistics and Actuarial Science, University of Waterloo, Tech. Rep. 2006-05, 2006.

[12] B. Bollobás, *Modern Graph Theory*, Graduate Texts in Mathematics, vol. 184, Springer, New York, NY, USA, 1998.

[13] M. Kula, "Metadata Embeddings for User and Item Cold-start Recommendations," in *Proceedings of the 2nd Workshop on New Trends on Content-Based Recommender Systems (CBRecSys)*, CEUR Workshop Proceedings, vol. 1448, 2015.

A Champion Clustering

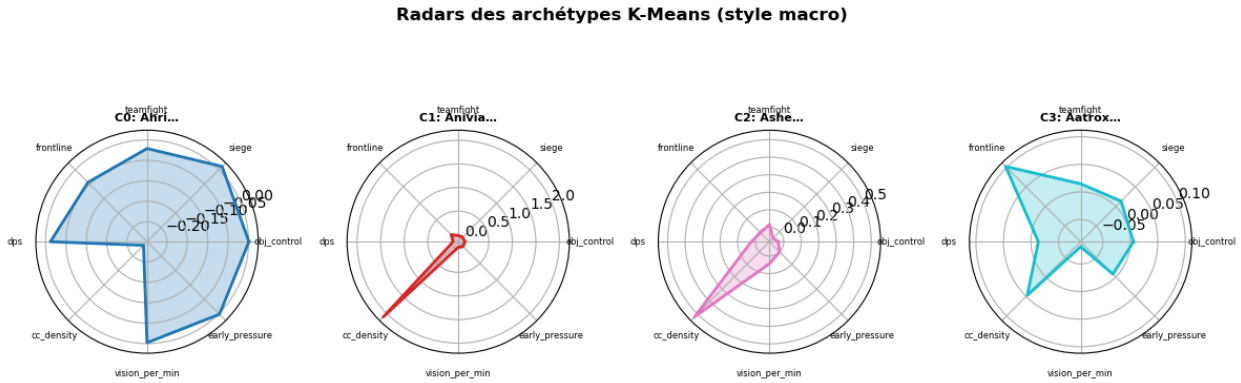


Figure 9: Radar plots of the four K-means champion archetypes (macro style vectors, $K = 4$). Each archetype is represented by its centroid champion (C0: Ahri, C1: Anivia, C2: Ashe, C3: Aatrox). The archetypes capture qualitatively distinct strategic profiles: C3 (Aatrox) shows high frontline, C1 (Anivia) high cc_density, and C2 (Ashe) a balanced DPS and siege profile.

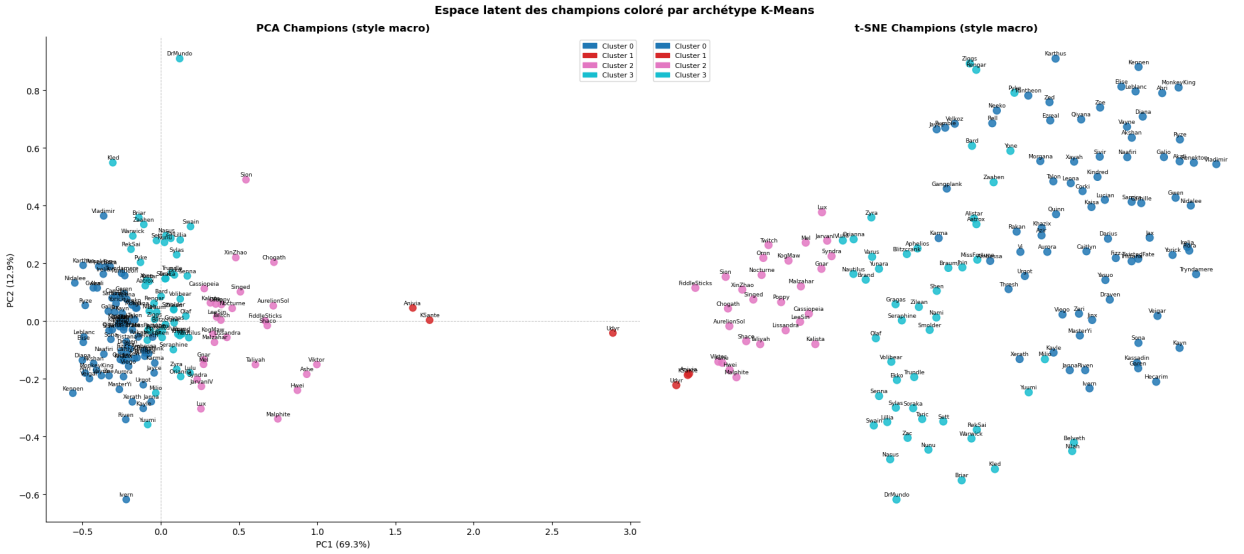


Figure 10: PCA (left) and t-SNE (right) projections of the 171 champions in macro style space, coloured by K-means cluster. The continuous latent space confirms that champion roles exist on a spectrum rather than as discrete categories. PC1 (69.3% variance) separates carry-oriented champions from support and tank profiles. Champions with strategic versatility (multi-role picks) consistently occupy intermediate positions.

B Win Rate and Popularity Correlation

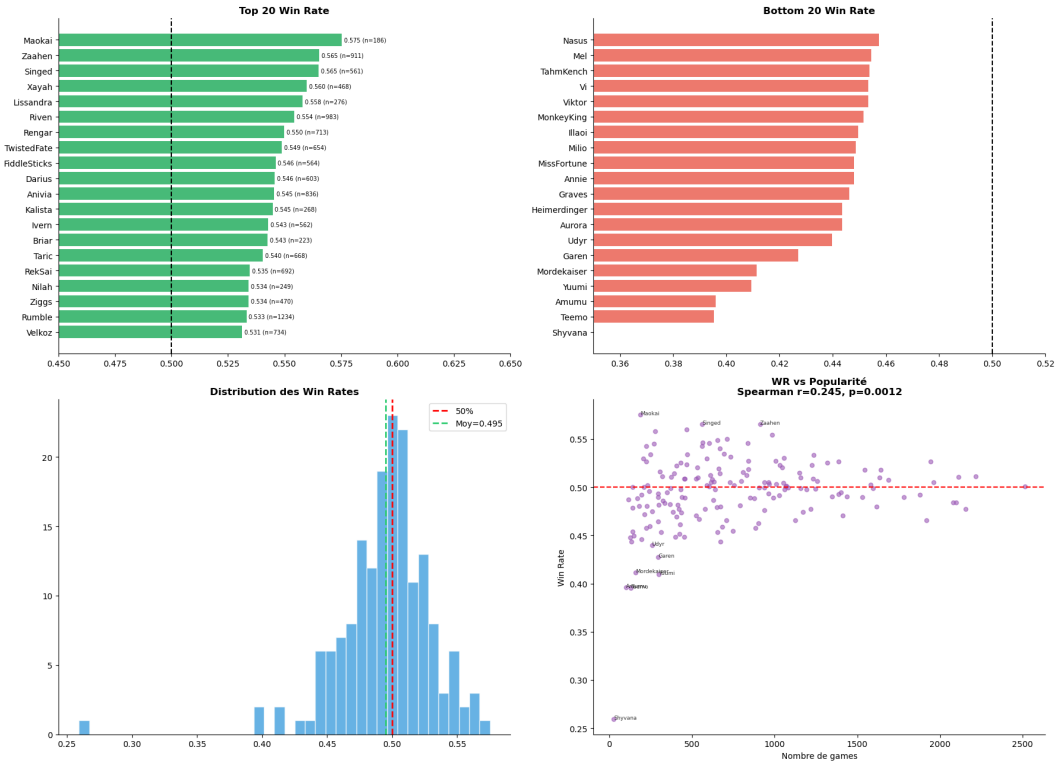


Figure 11: Champion win rate analysis. Top left: top 20 champions by win rate. Top right: bottom 20 champions by win rate. Bottom left: full distribution of individual win rates across 171 champions (mean = 0.495, close to 50%). Bottom right: scatter plot of win rate versus pick frequency (Spearman $r = 0.245$, $p = 0.0012$), demonstrating a statistically significant positive correlation between popularity and win rate. This empirically justifies the use of the LightFM uplift score to correct for popularity bias in synergy estimation.

C Full Model Comparison — Initial Feature Set

Table 5 reports AUC-ROC, accuracy, and training time for all classifiers evaluated on the initial feature set, prior to the V1–V4 feature engineering improvements, the LightFM uplift score, functional coverage, and the Katarina exclusion. These results served as a preliminary benchmark to guide architecture selection for the final evaluation.

Table 5: Full classifier benchmark on the initial feature set (∼17,000 soloQ games, before V1–V4 improvements). Results are not directly comparable to Table 2, which reports the final pipeline after all feature engineering improvements.

Model	**AUC**	**Accuracy**	**Time (s)**
Random Forest	0.5359	0.5229	2.7
Extra Trees	0.5357	0.5225	2.2
TabNet	0.5352	0.5248	487.6
MLP Large	0.5348	0.5239	7.0
Stacking	0.5343	0.5243	68.9
Voting (soft)	0.5337	0.5225	15.1
KNN ($k = 5$)	0.5333	0.5206	0.6
LightGBM Tuned	0.5315	0.5196	3.0
LightGBM	0.5315	0.5178	2.5
CatBoost	0.5310	0.5267	8.2
Logistic Regression	0.5304	0.5215	0.3
Ridge	0.5304	0.5196	0.2
Bagging LightGBM	0.5299	0.5210	30.3
SVM (RBF)	0.5296	0.5210	71.7
HistGradientBoosting	0.5284	0.5243	8.1
GradientBoosting	0.5271	0.5206	71.9
MLP	0.5264	0.5215	3.0
XGBoost	0.5252	0.5182	5.3
XGBoost DART	0.5251	0.5145	412.2
LR Polynomial (deg.= 2)	0.5158	0.4977	15.0